Синхронизированный доступ к списку

Родительская нить программы должна считывать вводимые пользователем строки и помещать их в начало связанного списка. Строки длиннее 80 символов можно разрезать на несколько строк. При вводе пустой строки программа должна выдавать текущее состояние списка.

Каждая дочерняя нить (их число настраивается) сортирует список в лексикографическом порядке бесконечным вариантом алгоритма пузырька (пузырек всегда пробегает весь список). При этом между шагами сортировки, а также внутри шагов должна происходить регулируемая задержка (например, в одну секунду). Под шагом понимается попытка поменять местами два элемента

При перестановке записей необходимо переустанавливать ссылки на записи, а не обменивать содержимое строк между записями. Стандартные реализации java.util.List не предоставляют публичных интерфейсов для реализации такого, вам необходимо самостоятельно реализовать одно- или двусвязный список. Класс списка должен имплементировать интерфейс Iterable, при выводе списка следует использовать стандартный цикл for в варианте for-each.

При задержке между шагами сортировки, пользователь должен немедленно видеть текущее (не сортированное или частично сортированное) состояние списка. При этом количество и содержимое записей в выведенном списке всегда должно соответствовать количеству и содержимому ранее введенных строк. Блокировка всего списка на время сортировки запрещена.

Для предотвращения мертвых блокировок записи, которые ближе к голове списка, всегда должны захватываться первыми.

Замените собственную реализацию списка на стандартный ArrayList с оберткой Collections.synchronizedList(). Адаптируйте реализацию сортировки, сохранив задержки.

Рассчитайте теоретически, сколько шагов алгоритма сортировки суммарно

сделают все дочерние нити в случаях с собственной реализацией списка и библиотечной. Измерьте фактическое количество шагов в обоих случаях и объясните результат.